Chapter 0

How this course works

practice teaches; the book shows you what to practise

You can already write Python: loops, functions, classes, a few sorting algorithms. The wall you hit is the next layer: a module you've never seen (`sqlite3`, `hmac`, `asyncio`), documentation that assumes you already know it, and programs big enough to need structure. This course is about that layer.

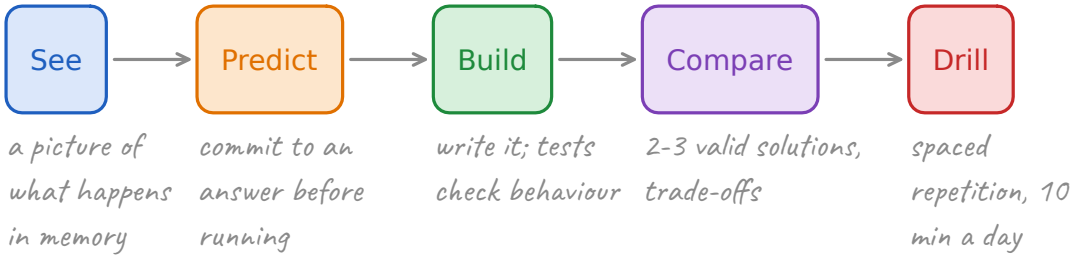
Text alone doesn't teach programming, any more than reading about press-ups builds muscle. So every page here is a **map of what to practise**, and the practice lives in a companion folder of exercises that check themselves. The book is short on purpose. The hours go into the exercises and into your own projects.

Three ideas run through everything:

- **The loop.** See it, predict it, build it, compare solutions, drill it.
- **The library card.** Six questions that crack open any library, in any language.
- **The language passport.** Twelve questions every language answers. Python fills them in now; C, Java and Rust fill in the same page later, so each new language is a diff, not a restart.

The first half of your Computing module (types, strings, lists, control flow, dictionaries, functions, classes) is squeezed into Chapter 1 as a fast recall sprint. NumPy gets a full chapter (18), because that's where the module gets interesting.

The loop: five steps, every topic



See. Every idea gets a picture: names pointing at objects, a resource opened and closed, bytes flowing through a pipeline, two threads on a timeline. The same pictures come back in every chapter, and later in every language.

Predict. Before running a snippet, write down what you think it prints. Being wrong is the useful part: it shows exactly where your mental model is off.

Build. The practice folder gives a task, an empty `solution.py` and a set of tests. The tests check what your code **does** (its contract), never how it looks, so any correct solution passes.

Compare. After you pass, you're shown two or three other solutions that also pass, with notes on speed, memory and readability. There's rarely one best answer; there's a best answer **for a situation**.

Drill. Ten minutes of quick questions a day ("what does this print?", "which module does X?"). Questions you get right come back less often; questions you miss come back tomorrow.

How to read a page

Code boxes. Grey boxes are code, with line numbers. Red handwriting points at the lines that matter. The dark box underneath is the **real output**: the book is built by running every snippet, so if a snippet printed something different, the book wouldn't build.

```
1 prices = {"tea": 1.20, "cake": 2.50}
2 order = ["tea", "cake", "tea"]
3 total = sum(prices[item] for item in order)
4 print(f"{len(order)} items, £{total:.2f}")
```

look each item up, add them

:.2f means 2 decimal places

```
3 items, £4.90
```

REPL boxes show an interactive session. Type the part after `>>>` yourself; Python echoes the value back. The REPL is your main tool for learning a library: poke it and see.

```
>>> word = "practice"
>>> word.upper()
'PRACTICE'
>>> len(word), word[::-1]
(8, 'ecitcarp')
```

Terminal boxes are commands typed in a shell (not run by the book):

```
$ cd Python-Visual-Guide
$ python -m practice next
```

KEY IDEA

Callout boxes like this one hold the one sentence to remember from a page. Red **TRAP** boxes mark the mistakes almost everyone makes once.

✎ YOUR GO (pencil, or the REPL, first)

Before reading on: what does print(sorted('banana')) print? Write it down, then try it.

Many right answers: tests check behaviour

Maths questions have one answer. Code usually has many: two sorting functions can look nothing alike and both be correct. So the practice tests never compare your code to mine. They check the **contract**: for a sort, "the output has the same items as the input, in order". They check it on hundreds of random inputs, including the awkward ones (empty, one item, all the same).

```python
import random

def is_sorted_version(out, original):
    same_items = sorted(out) == sorted(original)
    in_order = all(a <= b for a, b in zip(out, out[1:]))
    return same_items and in_order

def my_sort(xs):                # any correct sort would do
    xs = list(xs)
    for i in range(1, len(xs)):
        j = i
        while j and xs[j - 1] > xs[j]:
            xs[j - 1], xs[j] = xs[j], xs[j - 1]
            j -= 1
    return xs

rng = random.Random(0)
cases = [[], [7], [3, 3, 3]]
for _ in range(500):
    n = rng.randint(0, 12)
    cases.append([rng.randint(-9, 9) for _ in range(n)])
ok = all(is_sorted_version(my_sort(c), c) for c in cases)
print(ok, len(cases), "cases")
```

← same items...

← ...and in order

← the awkward cases first

```
True 503 cases
```

After you pass, the compare step shows other solutions side by side. Both of these pass the same tests; the bars are the trade-offs, not a ranking:

Insertion sort, by hand

```python
def sort_copy(xs):
    xs = list(xs)
    for i in range(1, len(xs)):
        j = i
        while j and xs[j-1] > xs[j]:
            xs[j-1], xs[j] = xs[j], xs[j-1]
            j -= 1
    return xs

print(sort_copy([5, 2, 9, 1]))
```

```
[1, 2, 5, 9]
```

Use the built-in

```python
def sort_copy(xs):
    return sorted(xs)


print(sort_copy([5, 2, 9, 1]))
```

```
[1, 2, 5, 9]
```

■□□□□	speed on big lists
■■■■■	teaches how sorting works
□□□■■	short and obviously right

RULE OF THUMB

Write it by hand to **learn** it; use the library to **ship** it. The exercises will often ask for both.

The library card: how to crack open any library

Getting stuck on `sqlite3` or `hmac` is rarely about syntax. It's about not knowing the **shape** of the library: which objects you create, which methods you call, what you must close, and what goes wrong. Every library, in every language, answers the same six questions. Chapter 7 teaches how to find the answers yourself; every later chapter fills in cards like this one:

LIBRARY CARD `json`

Job	Turn Python dicts/lists/str/numbers into JSON text and back.
Core objects	No objects to create: four functions. `dumps`/`loads` work on strings, `dump`/`load` on open files.
Lifecycle	data → `json.dumps(data)` → text → `json.loads(text)` → data.
Errors	`json.JSONDecodeError` for bad text; `TypeError` for things JSON can't hold (sets, dates).
Traps	Tuples come back as lists; dict keys always come back as strings.
Docs	docs.python.org/3/library/json.html, or `help(json)` in the REPL.

```
1 import json
2 data = {"id": 7, "tags": ("new", "hot"), 3: "three"}
3 text = json.dumps(data)
4 print(text)
5 print(json.loads(text))
```

← a tuple and an int key

← the traps, live

```
{"id": 7, "tags": ["new", "hot"], "3": "three"}
{'id': 7, 'tags': ['new', 'hot'], '3': 'three'}
```

The language passport

When you learn C, Java or Rust later, you won't start again. You'll answer the same twelve questions. Here is Python's passport in one page; each chapter ends with a small **stamp** filling in the questions it covered, and Appendix B collects them.

LANGUAGE PASSPORT: Python — same 12 questions for C, Java, Rust later

1. How does code run?
Source is compiled to bytecode, then run by the CPython virtual machine (Ch 2).

2. Memory model
Names point at objects on a heap; nothing is copied unless you ask (Ch 3).

3. Typing
Dynamic and strong: values have types, names don't; `'1' + 1` is an error. Optional hints (Ch 5, 17).

4. Control flow
`if`/`elif`, `for` over iterables, `while`, `match`; exceptions as non-local exits (Ch 1, 6).

5. Functions and scope
First-class functions, closures, LEGB scope, decorators, generators (Ch 4).

6. Data structures
list, tuple, dict, set built in; heapq, deque, bisect in the stdlib; know the costs (Ch 10, 20).

7. Abstraction
Classes with duck typing and protocols (dunder methods); dataclasses, ABCs (Ch 5).

8. Errors
Exceptions with `try`/`except`/`finally`; context managers for clean-up (Ch 6).

9. Modules and build
Modules and packages, `import`, venv + pip, `pyproject.toml` (Ch 2, 17, 19).

10. I/O
File objects, `pathlib`, text vs bytes with explicit encodings (Ch 8).

11. Concurrency
threads (GIL), processes, `asyncio`; free-threaded builds from 3.13 (Ch 16).

12. Stdlib and tooling
Huge stdlib ("batteries included"); `unittest`, `pdb`, `cProfile` built in (Appendix A).

Using the practice folder

The practice folder needs nothing but Python itself: no pip installs. From the course folder:

```
$ python -m practice list          # every exercise, and which you've passed
$ python -m practice next          # set up the next one and show the task
$ python -m practice check 12.3    # run the tests on your solution.py
$ python -m practice alt 12.3      # after a pass: other solutions + trade-offs
$ python -m practice drill         # today's spaced-repetition questions
```

Each exercise has a kind, so you know what's being asked:

kind	what you do
build from a spec	write it from nothing; the task says what it must do
fill the stub	the outline is there; fill in the missing parts
fix the bug	the code runs but is wrong; find and fix it
refactor	make it cleaner or faster without breaking the tests (sometimes with a rule, like "no loops")
learn an unseen module	use a module the book hasn't taught, with the library card method
measure and explain	time two versions with `timeit` and explain the difference
mini-project	a small real program that uses the whole chapter

A realistic day: 10 minutes of drill, 45 minutes of exercises, then the rest on your own project (your mini-uni infrastructure is perfect for this). The exercises give you the moves; the project is where they become habits.

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 0.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 0.1` shows other valid solutions and their trade-offs.

`0.1` build from a spec Warm-up: sort without sorted()

The map

part	chapters	what it unlocks
0 Recall	1 the module in one sprint	find your gaps fast
I Mental model	2 how Python runs · 3 names and memory · 4 functions · 5 the data model · 6 errors	stop being surprised by Python
II The unlock	7 how to learn any library	never stuck on a new module again
III Stdlib by job	8 files · 9 text · 10 algorithm toolkit · 11 numbers and time · 12 sqlite3 · 13 security · 14 OS and CLI · 15 networking · 16 concurrency · 17 quality	write most programs with no pip at all
IV Beyond	18 NumPy · 19 choosing pip packages	the outside world, safely
V Core CS	20 complexity and linked structures · 21 graphs and dynamic programming	the BSc algorithms module
Appendices	A stdlib atlas · B passport · C glossary	look things up

DEEP DIVE (your vault)

Your existing notes (the "Python and Computing Basics" vault) go far deeper on every topic. Each chapter points to the matching vault notes; read them when a page leaves you wanting the full story.